

APOSTILA PRÁTICA

OrderFlow

Pedidos, chat em tempo real e processamento assíncrono

C# / .NET 10

ASP.NET Core

PostgreSQL

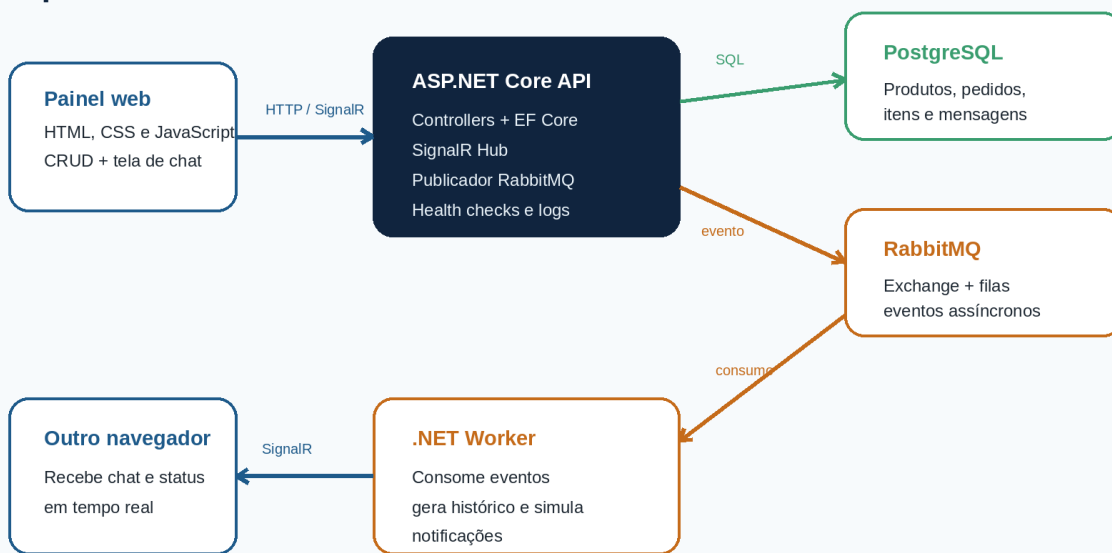
SignalR

RabbitMQ

Docker

Kubernetes

Arquitetura do OrderFlow



Docker empacota os componentes. Kubernetes executa, reinicia e escala os containers.

Ambiente principal: Windows 11 + PowerShell + Docker Desktop

Material preparado em julho de 2026

Como usar esta apostila

Esta apostila foi desenhada para ser executada, não apenas lida. Você construirá um projeto de portfólio em pequenas entregas, fará commits frequentes e terminará com uma demonstração pronta para GitHub e LinkedIn.

Resultado final

Uma aplicação chamada OrderFlow: catálogo de produtos, cadastro de clientes, criação e acompanhamento de pedidos, chat por pedido em tempo real e processamento assíncrono de eventos por um Worker .NET.

O que você aprenderá

- Organizar uma solução C#.NET em camadas sem transformar o estudo em uma arquitetura exagerada.
- Criar APIs REST com ASP.NET Core, Controllers, DTOs e Entity Framework Core.
- Modelar e persistir dados relacionais no PostgreSQL.
- Usar SignalR para chat e atualizações de status em tempo real.
- Usar RabbitMQ para separar tarefas lentas da requisição HTTP.
- Empacotar API, Worker, PostgreSQL e RabbitMQ com Docker.
- Executar o ambiente completo com Docker Compose.
- Publicar imagens e executar a aplicação em Kubernetes local.
- Criar testes, documentação, screenshots, vídeo e texto de divulgação.

Roteiro sugerido

Etapas	Entrega	Tempo sugerido
1	Preparar Windows 11, ferramentas e repositório	1 dia
2	Criar domínio, banco e migrations	2 dias
3	Implementar CRUD de produtos e clientes	2 dias
4	Criar pedidos, itens e regras de negócio	2 a 3 dias
5	Adicionar RabbitMQ e Worker	2 dias
6	Adicionar SignalR e chat por pedido	2 dias
7	Dockerizar e executar com Compose	2 dias
8	Testes, logs e tratamento de erros	2 dias
9	Implantar no Kubernetes local	2 a 3 dias
10	Finalizar README, vídeo e publicação	1 dia

Regra de estudo

Ao final de cada capítulo, execute o checklist e faça um commit. O histórico de commits mostra evolução real e aumenta a credibilidade do repositório.

1. Visão do projeto

Um sistema de pedidos com comunicação em tempo real e eventos assíncronos

1.1 A ideia: OrderFlow

O OrderFlow simula uma pequena operação de comércio ou restaurante. Um atendente cadastra produtos e clientes, cria pedidos, altera o status e conversa com o cliente dentro da sala daquele pedido. Quando eventos importantes acontecem, a API publica mensagens no RabbitMQ e um Worker .NET processa tarefas em segundo plano.

Funcionalidades do MVP

- CRUD de produtos: nome, descrição, preço, estoque e situação ativa/inativa.
- CRUD de clientes: nome e e-mail.
- Criação de pedido com um ou mais itens.
- Cálculo do total no backend e baixa de estoque.
- Fluxo de status: Criado, Confirmado, Em preparação, Enviado, Entregue ou Cancelado.
- Chat em tempo real separado por pedido.
- Histórico de mensagens persistido no PostgreSQL.
- Evento OrderCreated publicado no RabbitMQ.
- Worker que consome o evento e grava um registro de processamento.
- Ambiente completo em Docker Compose e Kubernetes.

Por que esse projeto chama atenção

Ele não é apenas mais um CRUD. O repositório demonstra três formas de comunicação comuns em sistemas profissionais: requisição HTTP síncrona, comunicação em tempo real e processamento assíncrono. Também mostra infraestrutura reproduzível, documentação e preocupação com falhas.

Três fluxos diferentes na mesma aplicação

1. Requisição síncrona



2. Tempo real



3. Processamento assíncrono



Figura 1 - A aplicação usa HTTP, SignalR e RabbitMQ para objetivos diferentes.

1.2 Arquitetura

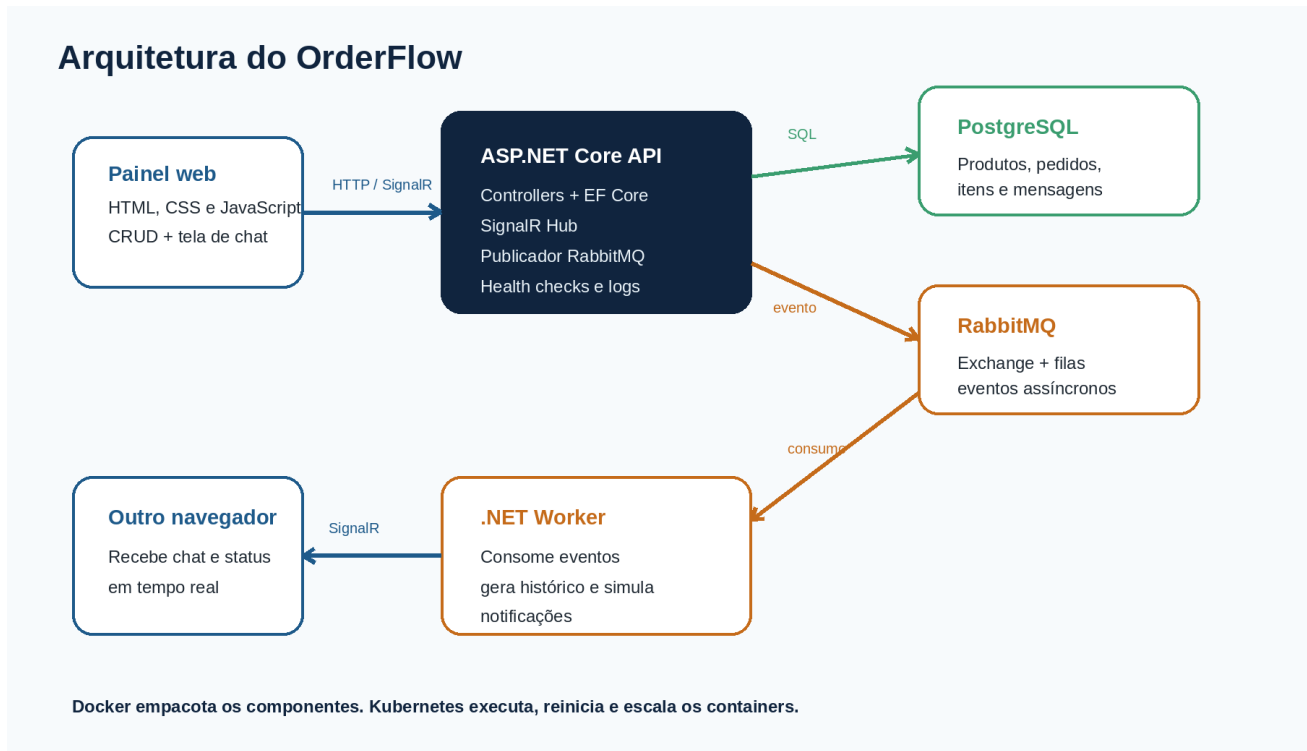


Figura 2 - Visão geral dos componentes do OrderFlow.

Importante

SignalR e RabbitMQ não competem. SignalR entrega informação instantaneamente a clientes conectados. RabbitMQ guarda e distribui trabalhos assíncronos entre processos.

1.3 Histórias de usuário

Perfil	História
Atendente	Quero cadastrar produtos para montar o catálogo.
Atendente	Quero criar um pedido escolhendo cliente, produtos e quantidades.
Atendente	Quero alterar o status e ver o cliente receber a atualização na hora.
Cliente	Quero acompanhar meu pedido e conversar com o atendimento.
Sistema	Quero processar eventos em segundo plano sem deixar a API lenta.
Desenvolvedor	Quero subir todo o ambiente com um único comando.

1.4 Critérios de conclusão

- ☐ Dois navegadores conseguem entrar na mesma sala e trocar mensagens.
- ☐ Uma mensagem de uma sala não aparece em outra sala.
- ☐ O histórico permanece após atualizar a página.
- ☐ A criação de pedido publica um evento no RabbitMQ.
- ☐ O Worker consome o evento e confirma a mensagem somente após processar.
- ☐ A aplicação roda com docker compose up -d.
- ☐ A aplicação também roda em um cluster Kubernetes local.

□ O GitHub possui README, diagrama, instruções, screenshots e testes.

2. Entendendo cada tecnologia

O papel de cada peça antes de escrever código

.NET 10: plataforma e runtime para executar aplicações C#. Desde o .NET 5, o nome “.NET Core” foi simplificado para “.NET”, enquanto ASP.NET Core e EF Core mantiveram “Core” no nome.

ASP.NET Core: framework web usado para criar a API HTTP, Controllers, middlewares, SignalR e endpoints de saúde.

Entity Framework Core: ORM que transforma operações com objetos C# em comandos SQL e permite criar migrations.

PostgreSQL: banco relacional que armazenará produtos, clientes, pedidos, itens, mensagens e histórico de eventos.

SignalR: biblioteca de tempo real. O servidor pode enviar mensagens a clientes conectados sem aguardar uma nova requisição HTTP.

RabbitMQ: broker de mensagens. A API publica eventos e o Worker os consome quando estiver disponível.

Docker: empacota cada componente com runtime, bibliotecas e configuração necessária para executá-lo.

Docker Compose: descreve e inicia vários containers que trabalham juntos em um ambiente local.

Kubernetes: orquestra containers: cria Pods, reinicia processos com falha, aplica configurações e permite escalar componentes.

Worker Service: processo .NET que fica ativo em segundo plano esperando mensagens do RabbitMQ.

2.1 Comparação rápida

Tecnologia	Pergunta que ela responde
.NET / ASP.NET Core	Como criar e executar a aplicação?
PostgreSQL	Onde os dados duráveis ficam armazenados?
SignalR	Como avisar clientes conectados imediatamente?
RabbitMQ	Como delegar um trabalho para ser feito depois?
Docker	Como executar a mesma aplicação em qualquer máquina?
Kubernetes	Como administrar vários containers de forma declarativa?

2.2 Conceitos que aparecerão no projeto

Conceito	Explicação simples
Container	Processo isolado criado a partir de uma imagem.
Imagem	Pacote imutável com aplicação e dependências.
Pod	Menor unidade executável do Kubernetes; normalmente contém um container principal.
Deployment	Declara quantas réplicas de uma aplicação devem existir.
Service	Fornece endereço estável para acessar Pods.
ConfigMap	Configuração não sensível, como hostname e nome de exchange.
Secret	Dados sensíveis, como senhas e connection strings.
Exchange	Recebe mensagens publicadas no RabbitMQ e as roteia.
Queue	Fila que guarda mensagens até um consumidor processá-las.
Ack	Confirmação de que uma mensagem foi processada.
Hub	Ponto central de comunicação do SignalR.
Group	Conjunto de conexões SignalR; no projeto, uma sala por pedido.

Princípio central

Use o banco para verdade durável, SignalR para experiência em tempo real e RabbitMQ para desacoplamento. Não dependa do SignalR para guardar histórico e não use RabbitMQ como banco de dados.

3. Preparando o Windows 11

Instalação e verificação das ferramentas

3.1 Ferramentas necessárias

Ferramenta	Uso
.NET 10 SDK	Compilar, testar e executar C#.
Git	Versionamento e GitHub.
VS Code ou Visual Studio	Editor/IDE.
Docker Desktop	Docker, Compose e Kubernetes local.
WSL 2	Backend Linux eficiente usado pelo Docker Desktop.
DBeaver ou pgAdmin	Opcional: visualizar o PostgreSQL.
Postman/Insomnia/REST Client	Opcional: testar a API.

3.2 Ativar e atualizar o WSL 2

Abra o PowerShell como administrador. Em uma instalação nova do Windows 11, o comando abaixo instala o WSL e uma distribuição Linux padrão. Reinicie o computador se solicitado.

PowerShell como administrador [PowerShell]

```
wsl --install
wsl --update
wsl --status
```

Opção mais simples

Você pode executar os comandos da apostila no PowerShell e guardar o projeto em C:\dev\OrderFlow. O WSL será usado pelo Docker nos bastidores. Depois, experimente desenvolver dentro do Ubuntu/WSL para conhecer o fluxo Linux.

3.3 Instalar e configurar o Docker Desktop

- Instale o Docker Desktop para Windows.
- Em Settings > General, mantenha o mecanismo baseado em WSL 2.
- Em Resources > WSL Integration, habilite a distribuição Ubuntu se for trabalhar dentro do WSL.
- Abra a área Kubernetes do Docker Desktop e crie um cluster local.
- Para um laboratório mais realista, selecione o provisionador kind e duas ou três máquinas; para economizar memória, use um nó.

Verificação [PowerShell]

```
docker version
docker compose version
kubectl version --client
kubectl config current-context
kubectl get nodes
```

3.4 Instalar e verificar o .NET 10 SDK

Verificação do SDK [PowerShell]

```
dotnet --info
```

```
dotnet --list-sdks
dotnet --version
```

A apostila usa .NET 10 por ser a versão LTS atual na data de preparação do material. Quando executar no futuro, use uma versão LTS suportada e mantenha a mesma versão principal entre SDK, pacotes, imagens Docker e documentação.

3.5 Extensões recomendadas no VS Code

- C# Dev Kit
- Docker
- Kubernetes
- REST Client
- PostgreSQL ou SQLTools
- YAML
- GitLens, opcional

3.6 Criar a pasta de trabalho

Criar o diretório e abrir o editor [PowerShell]

```
mkdir C:\dev
cd C:\dev
mkdir OrderFlow
cd OrderFlow
git init
code .
```

- ☐ dotnet --version começa com 10.
- ☐ docker version mostra Client e Server.
- ☐ docker compose version funciona.
- ☐ kubectl get nodes mostra pelo menos um nó Ready.
- ☐ O repositório Git está inicializado.

4. Criando a solução .NET

Estrutura de projetos, referências e pacotes

4.1 Criar os projetos

Comandos de criação [PowerShell]

```
dotnet new sln -n OrderFlow

mkdir src
mkdir tests

dotnet new classlib -n OrderFlow.Domain -o src/OrderFlow.Domain
dotnet new classlib -n OrderFlow.Application -o src/OrderFlow.Application
dotnet new classlib -n OrderFlow.Infrastructure -o src/OrderFlow.Infrastructure
dotnet new webapi -n OrderFlow.Api -o src/OrderFlow.Api --use-controllers
dotnet new worker -n OrderFlow.Worker -o src/OrderFlow.Worker
dotnet new xunit -n OrderFlow.Tests -o tests/OrderFlow.Tests

dotnet sln add src/OrderFlow.Domain/OrderFlow.Domain.csproj
dotnet sln add src/OrderFlow.Application/OrderFlow.Application.csproj
dotnet sln add src/OrderFlow.Infrastructure/OrderFlow.Infrastructure.csproj
dotnet sln add src/OrderFlow.Api/OrderFlow.Api.csproj
dotnet sln add src/OrderFlow.Worker/OrderFlow.Worker.csproj
dotnet sln add tests/OrderFlow.Tests/OrderFlow.Tests.csproj
```

4.2 Adicionar referências entre projetos

Referências [PowerShell]

```
dotnet add src/OrderFlow.Application reference src/OrderFlow.Domain
dotnet add src/OrderFlow.Infrastructure reference src/OrderFlow.Domain
dotnet add src/OrderFlow.Infrastructure reference src/OrderFlow.Application
dotnet add src/OrderFlow.Api reference src/OrderFlow.Application
dotnet add src/OrderFlow.Api reference src/OrderFlow.Infrastructure
dotnet add src/OrderFlow.Worker reference src/OrderFlow.Application
dotnet add src/OrderFlow.Worker reference src/OrderFlow.Infrastructure
dotnet add tests/OrderFlow.Tests reference src/OrderFlow.Domain
dotnet add tests/OrderFlow.Tests reference src/OrderFlow.Application
```

4.3 Instalar pacotes

Pacotes principais [PowerShell]

```
dotnet add src/OrderFlow.Infrastructure package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add src/OrderFlow.Infrastructure package Microsoft.EntityFrameworkCore.Design
dotnet add src/OrderFlow.Infrastructure package RabbitMQ.Client

dotnet add src/OrderFlow.Api package Swashbuckle.AspNetCore
dotnet add src/OrderFlow.Api package Microsoft.EntityFrameworkCore.Design

dotnet add src/OrderFlow.Worker package Microsoft.EntityFrameworkCore.Design
```

Sobre versões

Para estudo, instalar a versão estável compatível com o .NET 10 é suficiente. Em um projeto de produção, fixe versões no arquivo .csproj e use ferramentas de atualização controlada.

4.4 Estrutura final

Árvore do repositório [text]

```
OrderFlow/
|-- src/
|   |-- OrderFlow.Domain/           entidades, enums e regras centrais
|   |-- OrderFlow.Application/      DTOs, contratos e serviços de caso de uso
|   |-- OrderFlow.Infrastructure/    EF Core, PostgreSQL e RabbitMQ
|   |-- OrderFlow.Api/              Controllers, SignalR e interface web
|   |-- OrderFlow.Worker/           consumidor de mensagens
|-- tests/
|   |-- OrderFlow.Tests/
|-- docker-compose.yml
|-- k8s/
|-- docs/
|-- .github/workflows/
-- README.md
```

4.5 Criar o .gitignore e primeiro commit

Primeiro build [PowerShell]

```
dotnet new gitignore
dotnet restore
dotnet build
git add .
git commit -m "chore: create OrderFlow solution structure"
```

5. Modelando o domínio

Entidades, relacionamentos e regras de negócio

5.1 Enum de status

src/OrderFlow.Domain/Orders/OrderStatus.cs [C#]

```
namespace OrderFlow.Domain.Orders;

public enum OrderStatus
{
```



```

Created = 1,
Confirmed = 2,
Preparing = 3,
Shipped = 4,
Delivered = 5,
Cancelled = 6
}

```

5.2 Produto e cliente

src/OrderFlow.Domain/Products/Product.cs [C#]

```

namespace OrderFlow.Domain.Products;

public sealed class Product
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public string Name { get; private set; } = string.Empty;
    public string Description { get; private set; } = string.Empty;
    public decimal Price { get; private set; }
    public int Stock { get; private set; }
    public bool IsActive { get; private set; } = true;
    public DateTimeOffset CreatedAt { get; private set; } = DateTimeOffset.UtcNow;

    private Product() { }

    public Product(string name, string description, decimal price, int stock)
    {
        ChangeDetails(name, description, price);
        ChangeStock(stock);
    }

    public void ChangeDetails(string name, string description, decimal price)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Product name is required.");
        if (price <= 0)
            throw new ArgumentOutOfRangeException(nameof(price));

        Name = name.Trim();
        Description = description?.Trim() ?? string.Empty;
        Price = decimal.Round(price, 2);
    }

    public void ChangeStock(int stock)
    {
        if (stock < 0)
            throw new ArgumentOutOfRangeException(nameof(stock));
        Stock = stock;
    }

    public void RemoveFromStock(int quantity)
    {
        if (quantity <= 0 || quantity > Stock)
            throw new InvalidOperationException("Insufficient stock.");
        Stock -= quantity;
    }
}

```

src/OrderFlow.Domain/Customers/Customer.cs [C#]

```

namespace OrderFlow.Domain.Customers;

public sealed class Customer
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public string Name { get; private set; } = string.Empty;
    public string Email { get; private set; } = string.Empty;
    public DateTimeOffset CreatedAt { get; private set; } = DateTimeOffset.UtcNow;

    private Customer() { }

    public Customer(string name, string email)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException("Customer name is required.");
        if (string.IsNullOrEmpty(email) || !email.Contains('@'))
            throw new ArgumentException("A valid email is required.");

        Name = name.Trim();
        Email = email.Trim().ToLowerInvariant();
    }
}

```

}

5.3 Pedido e itens

src/OrderFlow.Domain/Orders/Order.cs [C#]

```
using OrderFlow.Domain.Customers;

namespace OrderFlow.Domain.Orders;

public sealed class Order
{
    private readonly List<OrderItem> _items = [];

    public Guid Id { get; private set; } = Guid.NewGuid();
    public Guid CustomerId { get; private set; }
    public Customer? Customer { get; private set; }
    public OrderStatus Status { get; private set; } = OrderStatus.Created;
    public decimal Total { get; private set; }
    public DateTimeOffset CreatedAt { get; private set; } = DateTimeOffset.UtcNow;
    public DateTimeOffset UpdatedAt { get; private set; } = DateTimeOffset.UtcNow;
    public IReadOnlyCollection<OrderItem> Items => _items;

    private Order() { }

    public Order(Guid customerId)
    {
        if (customerId == Guid.Empty)
            throw new ArgumentException("Customer is required.");
        CustomerId = customerId;
    }

    public void AddItem(Guid productId, string productName,
        int quantity, decimal unitPrice)
    {
        if (quantity <= 0) throw new ArgumentOutOfRangeException(nameof(quantity));
        if (unitPrice <= 0) throw new ArgumentOutOfRangeException(nameof(unitPrice));

        _items.Add(new OrderItem(Id, productId, productName, quantity, unitPrice));
        RecalculateTotal();
    }

    public void ChangeStatus(OrderStatus newStatus)
    {
        if (Status is OrderStatus.Delivered or OrderStatus.Cancelled)
            throw new InvalidOperationException("Finished order cannot be changed.");
        Status = newStatus;
        UpdatedAt = DateTimeOffset.UtcNow;
    }

    private void RecalculateTotal() => Total = _items.Sum(x => x.Subtotal);
}

public sealed class OrderItem
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public Guid OrderId { get; private set; }
    public Guid ProductId { get; private set; }
    public string ProductName { get; private set; } = string.Empty;
    public int Quantity { get; private set; }
    public decimal UnitPrice { get; private set; }
    public decimal Subtotal => Quantity * UnitPrice;

    private OrderItem() { }

    internal OrderItem(Guid orderId, Guid productId, string productName,
        int quantity, decimal unitPrice)
    {
        OrderId = orderId;
        ProductId = productId;
        ProductName = productName;
        Quantity = quantity;
        UnitPrice = unitPrice;
    }
}
```

5.4 Mensagens e histórico do Worker

src/OrderFlow.Domain/Chat/ChatMessage.cs e ProcessedEvent.cs [C#]

```
namespace OrderFlow.Domain.Chat;
```

```

public sealed class ChatMessage
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public Guid OrderId { get; private set; }
    public string Sender { get; private set; } = string.Empty;
    public string Text { get; private set; } = string.Empty;
    public DateTimeOffset SentAt { get; private set; } = DateTimeOffset.UtcNow;

    private ChatMessage() { }

    public ChatMessage(Guid orderId, string sender, string text)
    {
        if (orderId == Guid.Empty) throw new ArgumentException("Order is required.");
        if (string.IsNullOrEmpty(sender)) throw new ArgumentException("Sender is required.");
        if (string.IsNullOrEmpty(text)) throw new ArgumentException("Text is required.");
        if (text.Length > 1000) throw new ArgumentException("Text is too long.");

        OrderId = orderId;
        Sender = sender.Trim();
        Text = text.Trim();
    }
}

public sealed class ProcessedEvent
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public Guid EventId { get; private set; }
    public string EventType { get; private set; } = string.Empty;
    public string Payload { get; private set; } = string.Empty;
    public DateTimeOffset ProcessedAt { get; private set; } = DateTimeOffset.UtcNow;

    private ProcessedEvent() { }

    public ProcessedEvent(Guid eventId, string eventType, string payload)
    {
        EventId = eventId;
        EventType = eventType;
        Payload = payload;
    }
}

```

Decisão didática

As entidades protegem regras básicas dentro do domínio. Controllers não devem alterar propriedades diretamente. Isso evita que a regra fique espalhada em vários endpoints.

- ☐ O projeto Domain não referencia ASP.NET Core, EF Core ou RabbitMQ.
- ☐ O total do pedido é calculado pelo domínio.
- ☐ Preço e quantidade inválidos geram erro.
- ☐ Um pedido finalizado não aceita nova alteração de status.

6. PostgreSQL com Entity Framework Core

DbContext, configurações, migrations e índices

6.1 Criar o DbContext

src/OrderFlow.Infrastructure/Persistence/OrderFlowDbContext.cs [C#]

```

using Microsoft.EntityFrameworkCore;
using OrderFlow.Domain.Chat;
using OrderFlow.Domain.Customers;
using OrderFlow.Domain.Orders;
using OrderFlow.Domain.Products;

namespace OrderFlow.Infrastructure.Persistence;

public sealed class OrderFlowDbContext(DbContextOptions<OrderFlowDbContext> options)
    : DbContext(options)
{
    public DbSet<Product> Products => Set<Product>();
    public DbSet<Customer> Customers => Set<Customer>();
    public DbSet<Order> Orders => Set<Order>();
}

```

```

public DbSet<OrderItem> OrderItems => Set<OrderItem>();
public DbSet<ChatMessage> ChatMessages => Set<ChatMessage>();
public DbSet<ProcessedEvent> ProcessedEvents => Set<ProcessedEvent>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(typeof(OrderFlowDbContext).Assembly);
}
}

```

6.2 Configurar as entidades

ProductConfiguration.cs [C#]

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using OrderFlow.Domain.Products;

namespace OrderFlow.Infrastructure.Persistence.Configurations;

public sealed class ProductConfiguration : IEntityTypeConfiguration<Product>
{
    public void Configure(EntityTypeBuilder<Product> builder)
    {
        builder.ToTable("products");
        builder.HasKey(x => x.Id);
        builder.Property(x => x.Name).HasMaxLength(120).IsRequired();
        builder.Property(x => x.Description).HasMaxLength(500);
        builder.Property(x => x.Price).HasPrecision(12, 2);
        builder.HasIndex(x => x.Name);
    }
}

```

OrderConfiguration.cs [C#]

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using OrderFlow.Domain.Orders;

public sealed class OrderConfiguration : IEntityTypeConfiguration<Order>
{
    public void Configure(EntityTypeBuilder<Order> builder)
    {
        builder.ToTable("orders");
        builder.HasKey(x => x.Id);
        builder.Property(x => x.Status).HasConversion<string>().HasMaxLength(30);
        builder.Property(x => x.Total).HasPrecision(12, 2);

        builder.HasOne(x => x.Customer)
            .WithMany()
            .HasForeignKey(x => x.CustomerId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.HasMany(x => x.Items)
            .WithOne()
            .HasForeignKey(x => x.OrderId)
            .OnDelete(DeleteBehavior.Cascade);

        builder.Navigation(x => x.Items).UsePropertyAccessMode(PropertyAccessMode.Field);
        builder.HasIndex(x => new { x.CustomerId, x.CreatedAt });
    }
}

```

Crie configurações equivalentes para Customer, OrderItem, ChatMessage e ProcessedEvent. Para ProcessedEvent, crie um índice único em EventId; ele será usado para impedir processamento duplicado.

Índice de idempotência [C#]

```

builder.HasIndex(x => x.EventId).IsUnique();

```

6.3 Extensão de injeção de dependência

src/OrderFlow.Infrastructure/DependencyInjection.cs [C#]

```

using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using OrderFlow.Infrastructure.Persistence;

```

```
namespace OrderFlow.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString("Postgres")
            ?? throw new InvalidOperationException("Postgres connection string missing.");

        services.AddDbContext<OrderFlowDbContext>(options =>
            options.UseNpgsql(connectionString));

        return services;
    }
}
```

6.4 Configuração local

src/OrderFlow.Api/appsettings.Development.json [JSON]

```
{
  "ConnectionStrings": {
    "Postgres": "Host=localhost;Port=5432;Database=orderflow;Username=orderflow;Password=orderflow123"
  },
  "RabbitMq": {
    "Host": "localhost",
    "Port": 5672,
    "User": "orderflow",
    "Password": "orderflow123",
    "Exchange": "orderflow.events",
    "Queue": "orderflow.order-created"
  }
}
```

Segurança

A senha acima serve apenas para laboratório local. Não publique senhas reais no GitHub. Em produção, use variáveis de ambiente, Secret do Kubernetes ou um cofre de segredos.

6.5 Criar a migration

PowerShell/cmd: use uma linha se o caractere ^ não funcionar [PowerShell]

```
dotnet tool install --global dotnet-ef

dotnet ef migrations add InitialCreate ^
--project src/OrderFlow.Infrastructure ^
--startup-project src/OrderFlow.Api ^
--output-dir Persistence/Migrations

dotnet ef database update ^
--project src/OrderFlow.Infrastructure ^
--startup-project src/OrderFlow.Api
```

O banco ainda não estará disponível antes do Docker Compose. Você pode criar a migration agora e aplicar depois. O importante é entender que a migration registra as alterações de estrutura do banco como código versionado.

6.6 Índices importantes

- products(name): facilita busca por nome.
- orders(customer_id, created_at): facilita histórico de um cliente.
- chat_messages(order_id, sent_at): facilita carregar a conversa ordenada.
- processed_events(event_id) UNIQUE: impede processar o mesmo evento duas vezes.

7. Construindo a API REST

DTOs, Controllers e criação de pedidos

7.1 DTOs

src/OrderFlow.Application/DTOs.cs [C#]

```
namespace OrderFlow.Application.Products;

public sealed record CreateProductRequest(
    string Name,
    string Description,
    decimal Price,
    int Stock);

public sealed record ProductResponse(
    Guid Id,
    string Name,
    string Description,
    decimal Price,
    int Stock,
    bool IsActive);

public sealed record CreateCustomerRequest(string Name, string Email);

public sealed record CreateOrderItemRequest(Guid ProductId, int Quantity);
public sealed record CreateOrderRequest(
    Guid CustomerId,
    IReadOnlyCollection<CreateOrderItemRequest> Items);
```

7.2 ProdutosController

src/OrderFlow.Api/Controllers/ProductsController.cs [C#]

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using OrderFlow.Application.Products;
using OrderFlow.Domain.Products;
using OrderFlow.Infrastructure.Persistence;

namespace OrderFlow.Api.Controllers;

[ApiController]
[Route("api/products")]
public sealed class ProductsController(OrderFlowDbContext db) : ControllerBase
{
    [HttpGet]
    public async Task<ActionResult<IReadOnlyList<ProductResponse>>> GetAll(
        CancellationToken cancellationToken)
    {
        var products = await db.Products.AsNoTracking()
            .OrderBy(x => x.Name)
            .Select(x => new ProductResponse(
                x.Id, x.Name, x.Description, x.Price, x.Stock, x.IsActive))
            .ToListAsync(cancellationToken);

        return Ok(products);
    }

    [HttpPost]
    public async Task<ActionResult<ProductResponse>> Create(
        CreateProductRequest request,
        CancellationToken cancellationToken)
    {
        var product = new Product(
            request.Name, request.Description, request.Price, request.Stock);

        db.Products.Add(product);
        await db.SaveChangesAsync(cancellationToken);

        var response = new ProductResponse(product.Id, product.Name,
            product.Description, product.Price, product.Stock, product.IsActive);

        return CreatedAtAction(nameof(GetById), new { id = product.Id }, response);
    }

    [HttpGet("{id:guid}")]
```

```

public async Task<ActionResult<ProductResponse>> GetById(
    Guid id, CancellationToken cancellationToken)
{
    var product = await db.Products.AsNoTracking()
        .Where(x => x.Id == id)
        .Select(x => new ProductResponse(
            x.Id, x.Name, x.Description, x.Price, x.Stock, x.IsActive))
        .SingleOrDefaultAsync(cancellationToken);

    return product is null ? NotFound() : Ok(product);
}

```

Implemente PUT e DELETE lógico como exercício. No DELETE, desative o produto em vez de remover registros que podem ser referenciados por pedidos antigos.

7.3 Serviço de criação de pedido

src/OrderFlow.Application/Orders/OrderService.cs [C#]

```

using Microsoft.EntityFrameworkCore;
using OrderFlow.Domain.Orders;
using OrderFlow.Infrastructure.Messaging;
using OrderFlow.Infrastructure.Persistence;

namespace OrderFlow.Application.Orders;

public sealed class OrderService(
    OrderFlowDbContext db,
    IEventPublisher eventPublisher)
{
    public async Task<Guid> CreateAsync(
        CreateOrderRequest request,
        CancellationToken cancellationToken)
    {
        if (request.Items.Count == 0)
            throw new InvalidOperationException("Order must contain items.");

        var customerExists = await db.Customers
            .AnyAsync(x => x.Id == request.CustomerId, cancellationToken);
        if (!customerExists)
            throw new KeyNotFoundException("Customer not found.");

        var productIds = request.Items.Select(x => x.ProductId).Distinct().ToList();
        var products = await db.Products
            .Where(x => productIds.Contains(x.Id) && x.IsActive)
            .ToDictionaryAsync(x => x.Id, cancellationToken);

        await using var transaction = await db.Database
            .BeginTransactionAsync(cancellationToken);

        var order = new Order(request.CustomerId);
        foreach (var item in request.Items)
        {
            if (!products.TryGetValue(item.ProductId, out var product))
                throw new KeyNotFoundException($"Product {item.ProductId} not found.");

            product.RemoveFromStock(item.Quantity);
            order.AddItem(product.Id, product.Name, item.Quantity, product.Price);
        }

        db.Orders.Add(order);
        await db.SaveChangesAsync(cancellationToken);
        await transaction.CommitAsync(cancellationToken);

        await eventPublisher.PublishAsync(
            "order.created",
            new OrderCreatedEvent(
                Guid.NewGuid(), order.Id, order.CustomerId,
                order.Total, DateTimeOffset.UtcNow,
                cancellationToken);

        return order.Id;
    }
}

```

Ponto de arquitetura

Salvar no banco e publicar no RabbitMQ são duas operações diferentes. No MVP, publicaremos após o commit. Em uma evolução profissional, implemente o padrão Outbox para evitar a situação em que o banco confirma e a publicação falha.

7.4 Endpoints planejados

Método	Rota	Objetivo
GET	/api/products	Listar produtos
POST	/api/products	Criar produto
PUT	/api/products/{id}	Atualizar produto
DELETE	/api/products/{id}	Desativar produto
GET	/api/customers	Listar clientes
POST	/api/customers	Criar cliente
POST	/api/orders	Criar pedido
GET	/api/orders/{id}	Detalhar pedido e itens
PATCH	/api/orders/{id}/status	Alterar status
GET	/api/orders/{id}/messages	Carregar histórico do chat

7.5 Program.cs da API

src/OrderFlow.Api/Program.cs [C#]

```
using OrderFlow.Application.Orders;
using OrderFlow.Api.Hubs;
using OrderFlow.Infrastructure;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddSignalR();
builder.Services.AddInfrastructure(builder.Configuration);
builder.Services.AddScoped<OrderService>();
builder.Services.AddHealthChecks();

var app = builder.Build();

app.UseDefaultFiles();
app.UseStaticFiles();
app.UseSwagger();
app.UseSwaggerUI();
app.MapControllers();
app.MapHub<OrderChatHub>("/hubs/orders");
app.MapHealthChecks("/health");
app.Run();
```

7.6 Teste rápido com arquivo .http

requests/OrderFlow.http [HTTP]

```
@baseUrl = http://localhost:8080

### Criar produto
POST {{baseUrl}}/api/products
Content-Type: application/json

{
  "name": "Teclado Mecânico",
  "description": "Switch marrom, ABNT2",
  "price": 349.90,
  "stock": 20
}

### Listar produtos
GET {{baseUrl}}/api/products
```


8. Mensageria com RabbitMQ

Publicação, filas, Worker, ack e idempotência

8.1 Topologia

Objeto	Nome	Função
Exchange topic	orderflow.events	Recebe eventos da API.
Routing key	order.created	Identifica evento de pedido criado.
Queue durable	orderflow.order-created	Armazena eventos para o Worker.
Binding	order.created	Liga a fila ao exchange.

8.2 Contrato do evento

OrderCreatedEvent.cs [C#]

```
namespace OrderFlow.Application.Orders;

public sealed record OrderCreatedEvent(
    Guid EventId,
    Guid OrderId,
    Guid CustomerId,
    decimal Total,
    DateTimeOffset OccurredAt);
```

8.3 Interface e opções

Messaging contracts [C#]

```
public interface IEventPublisher
{
    Task PublishAsync<T>(
        string routingKey,
        T message,
        CancellationToken cancellationToken = default);
}

public sealed class RabbitMqOptions
{
    public string Host { get; init; } = "localhost";
    public int Port { get; init; } = 5672;
    public string User { get; init; } = "guest";
    public string Password { get; init; } = "guest";
    public string Exchange { get; init; } = "orderflow.events";
    public string Queue { get; init; } = "orderflow.order-created";
}
```

8.4 Publicador RabbitMQ.Client 7

src/OrderFlow.Infrastructure/Messaging/RabbitMqPublisher.cs [C#]

```
using System.Text;
using System.Text.Json;
using Microsoft.Extensions.Options;
using RabbitMQ.Client;

namespace OrderFlow.Infrastructure.Messaging;

public sealed class RabbitMqPublisher : IEventPublisher, IAsyncDisposable
{
    private readonly RabbitMqOptions _options;
    private readonly SemaphoreSlim _publishLock = new(1, 1);
    private IConnection? _connection;
    private IChannel? _channel;

    public RabbitMqPublisher(IOptions<RabbitMqOptions> options)
        => _options = options.Value;

    private async Task EnsureConnectedAsync()
    {

```

```

        if (_connection?.IsOpen == true && _channel?.IsOpen == true) return;

        var factory = new ConnectionFactory
        {
            HostName = _options.Host,
            Port = _options.Port,
            UserName = _options.User,
            Password = _options.Password,
            AutomaticRecoveryEnabled = true,
            ClientProvidedName = "orderflow-api-publisher"
        };

        _connection = await factory.CreateConnectionAsync();
        _channel = await _connection.CreateChannelAsync();
        await _channel.ExchangeDeclareAsync(
            _options.Exchange, ExchangeType.Topic, durable: true);
    }

    public async Task PublishAsync<T>(string routingKey, T message,
        CancellationToken cancellationToken = default)
    {
        await EnsureConnectedAsync();
        await _publishLock.WaitAsync(cancellationToken);
        try
        {
            var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(message));
            var properties = new BasicProperties
            {
                ContentType = "application/json",
                DeliveryMode = DeliveryModes.Persistent,
                MessageId = Guid.NewGuid().ToString()
            };

            await _channel!.BasicPublishAsync(
                _options.Exchange,
                routingKey,
                mandatory: false,
                basicProperties: properties,
                body: body,
                cancellationToken: cancellationToken);
        }
        finally
        {
            _publishLock.Release();
        }
    }

    public async ValueTask DisposeAsync()
    {
        if (_channel is not null) await _channel.DisposeAsync();
        if (_connection is not null) await _connection.DisposeAsync();
        _publishLock.Dispose();
    }
}

```

Por que conexão longa?

Conexões e canais RabbitMQ foram feitos para durar. Abrir uma conexão para cada mensagem é caro. O semáforo impede publicações concorrentes no mesmo canal.

8.5 Registrar o publicador

Adicionar em `DependencyInjection.cs` [C#]

```

services.Configure<RabbitMQOptions>(configuration.GetSection("RabbitMQ"));
services.AddSingleton<IEventPublisher, RabbitMqPublisher>();

```

8.6 Worker consumidor

`src/OrderFlow.Worker/OrderCreatedWorker.cs` [C#]

```

using System.Text;
using System.Text.Json;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Options;
using OrderFlow.Application.Orders;
using OrderFlow.Domain.Chat;
using OrderFlow.Infrastructure.Messaging;
using OrderFlow.Infrastructure.Persistence;

```

```

using RabbitMQ.Client;
using RabbitMQ.Client.Events;

namespace OrderFlow.Worker;

public sealed class OrderCreatedWorker(
    IServiceScopeFactory scopeFactory,
    IOptions<RabbitMqOptions> options,
    ILogger<OrderCreatedWorker> logger) : BackgroundService
{
    private IConnection? _connection;
    private IChannel? _channel;
    private readonly RabbitMqOptions _options = options.Value;

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        var factory = new ConnectionFactory
        {
            HostName = _options.Host,
            Port = _options.Port,
            UserName = _options.User,
            Password = _options.Password,
            AutomaticRecoveryEnabled = true,
            ClientProvidedName = "orderflow-order-created-worker"
        };

        _connection = await factory.CreateConnectionAsync(stoppingToken);
        _channel = await _connection.CreateChannelAsync(cancellationToken: stoppingToken);

        await _channel.ExchangeDeclareAsync(
            _options.Exchange, ExchangeType.Topic, durable: true,
            cancellationToken: stoppingToken);
        await _channel.QueueDeclareAsync(
            _options.Queue, durable: true, exclusive: false,
            autoDelete: false, cancellationToken: stoppingToken);
        await _channel.QueueBindAsync(
            _options.Queue, _options.Exchange, "order.created",
            cancellationToken: stoppingToken);
        await _channel.BasicQosAsync(0, 5, false, stoppingToken);

        var consumer = new AsyncEventingBasicConsumer(_channel);
        consumer.ReceivedAsync += HandleMessageAsync;

        await _channel.BasicConsumeAsync(
            _options.Queue, autoAck: false, consumer,
            cancellationToken: stoppingToken);

        await Task.Delay(Timeout.Infinite, stoppingToken);
    }

    private async Task HandleMessageAsync(object sender, BasicDeliverEventArgs args)
    {
        var body = args.Body.ToArray();
        var json = Encoding.UTF8.GetString(body);

        try
        {
            var message = JsonSerializer.Deserialize<OrderCreatedEvent>(json)
                ?? throw new JsonException("Invalid event payload.");

            await using var scope = scopeFactory.CreateAsyncScope();
            var db = scope.ServiceProvider.GetRequiredService<OrderFlowDbContext>();

            var alreadyProcessed = await db.ProcessedEvents
                .AnyAsync(x => x.EventId == message.EventId);

            if (!alreadyProcessed)
            {
                logger.LogInformation(
                    "Processing order {OrderId} with total {Total}",
                    message.OrderId, message.Total);

                await Task.Delay(800); // simula email, nota ou integração
                db.ProcessedEvents.Add(new ProcessedEvent(
                    message.EventId, nameof(OrderCreatedEvent), json));
                await db.SaveChangesAsync();
            }

            await _channel!.BasicAckAsync(args.DeliveryTag, multiple: false);
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "Error processing message {DeliveryTag}", args.DeliveryTag);
            await _channel!.BasicNackAsync(
                args.DeliveryTag, multiple: false, requeue: false);
        }
    }
}

```

```

    }
}

public override async Task StopAsync(CancellationToken cancellationToken)
{
    if (_channel is not null) await _channel.DisposeAsync();
    if (_connection is not null) await _connection.DisposeAsync();
    await base.StopAsync(cancellationToken);
}
}

```

8.7 Program.cs do Worker

src/OrderFlow.Worker/Program.cs [C#]

```

using OrderFlow.Infrastructure;
using OrderFlow.Worker;

var builder = Host.CreateApplicationBuilder(args);
builder.Services.AddInfrastructure(builder.Configuration);
builder.Services.AddHostedService<OrderCreatedWorker>();

var host = builder.Build();
host.Run();

```

8.8 Ack, Nack, retry e DLQ

- Ack: o Worker processou com sucesso e o RabbitMQ pode remover a mensagem.
- Nack com requeue false: a mensagem falhou e não volta imediatamente para a fila.
- Em uma evolução, configure uma Dead Letter Queue para guardar mensagens problemáticas.
- Evite requeue infinito, pois uma mensagem inválida pode consumir CPU sem parar.
- Idempotência significa que receber o mesmo evento duas vezes não cria dois efeitos.

- ☐ O exchange e a fila aparecem no painel do RabbitMQ.
- ☐ Criar um pedido aumenta a taxa de publicação.
- ☐ O Worker registra o evento em processed_events.
- ☐ Reenviar o mesmo EventId não gera duplicidade.

9. Chat em tempo real com SignalR

Salas por pedido, persistência e atualização de status

9.1 Como a sala funciona

Cada conexão entra em um grupo cujo nome contém o ID do pedido. Quando uma mensagem é enviada, o Hub salva no PostgreSQL e transmite apenas ao grupo daquele pedido. Assim, pedidos diferentes ficam isolados.

9.2 Hub

src/OrderFlow.Api/Hubs/OrderChatHub.cs [C#]

```

using Microsoft.AspNetCore.SignalR;
using Microsoft.EntityFrameworkCore;
using OrderFlow.Domain.Chat;
using OrderFlow.Infrastructure.Persistence;

namespace OrderFlow.Api.Hubs;

public sealed class OrderChatHub(OrderFlowDbContext db) : Hub
{
    private static string GroupName(Guid orderId) => $"order:{orderId}";

    public async Task JoinOrder(Guid orderId)
    {
        var exists = await db.Orders.AnyAsync(x => x.Id == orderId);
        if (!exists) throw new HubException("Order not found.");
    }
}

```

```

        await Groups.AddToGroupAsync(Context.ConnectionId, GroupName(orderId));
    }

    public async Task LeaveOrder(Guid orderId)
    => await Groups.RemoveFromGroupAsync(
        Context.ConnectionId, GroupName(orderId));

    public async Task SendMessage(Guid orderId, string sender, string text)
    {
        var exists = await db.Orders.AnyAsync(x => x.Id == orderId);
        if (!exists) throw new HubException("Order not found.");

        var message = new ChatMessage(orderId, sender, text);
        db.ChatMessages.Add(message);
        await db.SaveChangesAsync();

        await Clients.Group(GroupName(orderId)).SendAsync(
            "MessageReceived",
            new
            {
                message.Id,
                message.OrderId,
                message.Sender,
                message.Text,
                message.SentAt
            });
    }
}

```

Não guarde estado no Hub

Instâncias de Hub são transitórias. O histórico fica no PostgreSQL; o grupo é administrado pelo SignalR; dados permanentes nunca devem ficar em propriedades do Hub.

9.3 Histórico por HTTP

Endpoint no OrdersController [C#]

```

[HttpGet("{orderId:guid}/messages")]
public async Task<IActionResult> GetMessages(
    Guid orderId, CancellationToken cancellationToken)
{
    var messages = await db.ChatMessages.AsNoTracking()
        .Where(x => x.OrderId == orderId)
        .OrderBy(x => x.SentAt)
        .Select(x => new { x.Id, x.Sender, x.Text, x.SentAt })
        .ToListAsync(cancellationToken);

    return Ok(messages);
}

```

9.4 Broadcast de mudança de status

IHubContext permite enviar fora do Hub [C#]

```

public sealed class OrdersController(
    OrderFlowDbContext db,
    IHubContext<OrderChatHub> hubContext) : ControllerBase
{
    [HttpPatch("{id:guid}/status")]
    public async Task<IActionResult> ChangeStatus(
        Guid id, ChangeOrderStatusRequest request,
        CancellationToken cancellationToken)
    {
        var order = await db.Orders.FindAsync([id], cancellationToken);
        if (order is null) return NotFound();

        order.ChangeStatus(request.Status);
        await db.SaveChangesAsync(cancellationToken);

        await hubContext.Clients.Group($"order:{id}").SendAsync(
            "StatusChanged",
            new { orderId = id, status = order.Status.ToString() },
            cancellationToken);

        return NoContent();
    }
}

```

9.5 Página de demonstração

Crie a pasta `src/OrderFlow.Api/wwwroot` e um arquivo `index.html`. A página abaixo é propositalmente simples: o objetivo principal é demonstrar o backend. Depois, você pode melhorar o visual ou trocar por React, Angular ou Blazor.

`src/OrderFlow.Api/wwwroot/index.html` [HTML]

```

<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>OrderFlow - Chat do pedido</title>
  <style>
    body { font-family: Arial; background:#f3f5f7; margin:0; }
    main { max-width:900px; margin:40px auto; background:white; padding:24px;
      border-radius:16px; box-shadow:0 10px 30px #0001; }
    input, button { padding:12px; margin:4px; }
    #messages { min-height:260px; border:1px solid #ddd; padding:16px;
      border-radius:10px; overflow:auto; }
    .message { padding:8px 12px; margin:6px 0; background:#eaf3f9;
      border-radius:10px; }
  </style>
</head>
<body>
<main>
  <h1>OrderFlow</h1>
  <p>Entre em uma sala usando o ID do pedido.</p>
  <input id="orderId" placeholder="ID do pedido" size="38">
  <input id="sender" placeholder="Seu nome" value="Atendente">
  <button id="connect">Conectar</button>
  <p id="status">Desconectado</p>
  <div id="messages"></div>
  <input id="text" placeholder="Digite a mensagem" size="55">
  <button id="send">Enviar</button>
</main>
<script src="https://cdn.jsdelivr.net/npm/@microsoft/signalr@10/dist/browser/signalr.min.js"></script>
<script>
let connection;
const messages = document.getElementById('messages');

function addMessage(text) {
  const div = document.createElement('div');
  div.className = 'message';
  div.textContent = text;
  messages.appendChild(div);
}

document.getElementById('connect').onclick = async () => {
  const orderId = document.getElementById('orderId').value;
  connection = new signalR.HubConnectionBuilder()
    .withUrl('/hubs/orders')
    .withAutomaticReconnect()
    .build();

  connection.on('MessageReceived', m =>
    addMessage(`${m.sender}: ${m.text}`));
  connection.on('StatusChanged', s =>
    addMessage(`STATUS: ${s.status}`));

  await connection.start();
  await connection.invoke('JoinOrder', orderId);
  document.getElementById('status').textContent = 'Conectado';
};

document.getElementById('send').onclick = async () => {
  await connection.invoke(
    'SendMessage',
    document.getElementById('orderId').value,
    document.getElementById('sender').value,
    document.getElementById('text').value);
  document.getElementById('text').value = '';
};
</script>
</body>
</html>

```

9.6 Teste em duas janelas

- Crie um pedido pela API e copie o ID.
- Abra <http://localhost:8080> em duas janelas ou em modo anônimo.
- Use o mesmo ID do pedido e nomes diferentes.
- Envie mensagens nos dois sentidos.
- Altere o status pela API e veja o aviso surgir nas duas janelas.
- Atualize a página e carregue o histórico pelo endpoint HTTP.

10. Docker e Docker Compose

Empacotando e iniciando todo o ambiente

10.1 Dockerfile da API

`src/OrderFlow.Api/Dockerfile` [Dockerfile]

```
# syntax=docker/dockerfile:1
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore OrderFlow.sln
RUN dotnet publish src/OrderFlow.Api/OrderFlow.Api.csproj \
    -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "OrderFlow.Api.dll"]
```

10.2 Dockerfile do Worker

`src/OrderFlow.Worker/Dockerfile` [Dockerfile]

```
# syntax=docker/dockerfile:1
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore OrderFlow.sln
RUN dotnet publish src/OrderFlow.Worker/OrderFlow.Worker.csproj \
    -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/runtime:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "OrderFlow.Worker.dll"]
```

10.3 .dockerignore

`.dockerignore` [text]

```
**/bin/
**/obj/
.git/
.vs/
.vscode/
TestResults/
coverage/
README.md
```

10.4 docker-compose.yml

`docker-compose.yml` [YAML]

```
services:
  postgres:
```

```

image: postgres:18-alpine
environment:
  POSTGRES_DB: orderflow
  POSTGRES_USER: orderflow
  POSTGRES_PASSWORD: orderflow123
ports:
  - "5432:5432"
volumes:
  - postgres_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U orderflow -d orderflow"]
  interval: 5s
  timeout: 5s
  retries: 10

rabbitmq:
image: rabbitmq:4-management-alpine
environment:
  RABBITMQ_DEFAULT_USER: orderflow
  RABBITMQ_DEFAULT_PASS: orderflow123
ports:
  - "5672:5672"
  - "15672:15672"
volumes:
  - rabbitmq_data:/var/lib/rabbitmq
healthcheck:
  test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
  interval: 10s
  timeout: 5s
  retries: 10

api:
build:
  context: .
  dockerfile: src/OrderFlow.Api/Dockerfile
environment:
  ASPNETCORE_ENVIRONMENT: Development
  ConnectionStrings__Postgres: >-
    Host=postgres;Port=5432;Database=orderflow;
    Username=orderflow;Password=orderflow123
  RabbitMq__Host: rabbitmq
  RabbitMq__Port: 5672
  RabbitMq__User: orderflow
  RabbitMq__Password: orderflow123
  RabbitMq__Exchange: orderflow.events
  RabbitMq__Queue: orderflow.order-created
ports:
  - "8080:8080"
depends_on:
  postgres:
    condition: service_healthy
  rabbitmq:
    condition: service_healthy

worker:
build:
  context: .
  dockerfile: src/OrderFlow.Worker/Dockerfile
environment:
  ConnectionStrings__Postgres: >-
    Host=postgres;Port=5432;Database=orderflow;
    Username=orderflow;Password=orderflow123
  RabbitMq__Host: rabbitmq
  RabbitMq__Port: 5672
  RabbitMq__User: orderflow
  RabbitMq__Password: orderflow123
  RabbitMq__Exchange: orderflow.events
  RabbitMq__Queue: orderflow.order-created
depends_on:
  postgres:
    condition: service_healthy
  rabbitmq:
    condition: service_healthy

volumes:
  postgres_data:
  rabbitmq_data:

```

10.5 Aplicar migrations automaticamente no laboratório

Para estudo, você pode aplicar migrations ao iniciar a API. Em produção, prefira um job de migração controlado no pipeline.

Adicionar antes de app.Run() no Program.cs [C#]

```
using Microsoft.EntityFrameworkCore;
using OrderFlow.Infrastructure.Persistence;

await using (var scope = app.Services.CreateAsyncScope())
{
    var db = scope.ServiceProvider.GetRequiredService<OrderFlowDbContext>();
    await db.Database.MigrateAsync();
}
```

10.6 Executar

Subir o laboratório [PowerShell]

```
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f api worker
```

Recurso	Endereço
Aplicação/chat	http://localhost:8080
Swagger	http://localhost:8080/swagger
Health check	http://localhost:8080/health
RabbitMQ Management	http://localhost:15672
PostgreSQL	localhost:5432

10.7 Comandos de diagnóstico

Diagnóstico [PowerShell]

```
docker compose ps
docker compose logs --tail 100 api
docker compose logs --tail 100 worker
docker compose exec postgres psql -U orderflow -d orderflow
docker compose exec rabbitmq rabbitmqctl list_queues name messages consumers
docker compose restart worker
docker compose down
docker compose down -v # também remove os dados; use com cuidado
```

DNS interno do Compose

Dentro dos containers, localhost significa o próprio container. A API conecta em postgres e rabbitmq usando os nomes dos serviços do Compose.

11. Testes e qualidade

Regras de domínio, integração e observabilidade básica

11.1 Testes unitários

tests/OrderFlow.Tests/OrderTests.cs [C#]

```
using OrderFlow.Domain.Orders;
using OrderFlow.Domain.Products;

namespace OrderFlow.Tests;

public sealed class OrderTests
{
    [Fact]
    public void AddItem_ShouldCalculateTotal()
    {
        var order = new Order(Guid.NewGuid());
        order.AddItem(Guid.NewGuid(), "Mouse", 2, 99.90m);
        order.AddItem(Guid.NewGuid(), "Teclado", 1, 300m);
    }
}
```

```

        Assert.Equal(499.80m, order.Total);
        Assert.Equal(2, order.Items.Count);
    }

    [Fact]
    public void Product_ShouldRejectInsufficientStock()
    {
        var product = new Product("Mouse", "Sem fio", 100m, 2);

        Assert.Throws<InvalidOperationException>(() =>
            product.RemoveFromStock(3));
    }

    [Fact]
    public void FinishedOrder_ShouldRejectStatusChange()
    {
        var order = new Order(Guid.NewGuid());
        order.ChangeStatus(OrderStatus.Delivered);

        Assert.Throws<InvalidOperationException>(() =>
            order.ChangeStatus(OrderStatus.Cancelled));
    }
}

```

Executar testes [PowerShell]

```

dotnet test
dotnet test --collect:"XPlat Code Coverage"

```

11.2 Testes de integração como evolução

- Subir PostgreSQL e RabbitMQ reais em containers durante o teste.
- Criar um pedido pela API e verificar a tabela orders.
- Verificar se o evento chegou à fila.
- Conectar dois clientes SignalR e confirmar a entrega da mensagem.
- Testar que uma mensagem de um pedido não vaza para outro grupo.

11.3 Tratamento global de erros

Middleware simples para Problem Details [C#]

```

app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        context.Response.ContentType = "application/problem+json";
        var feature = context.Features
            .Get<Microsoft.AspNetCore.Diagnostics.IExceptionHandlerFeature>();

        context.Response.StatusCode = feature?.Error switch
        {
            KeyNotFoundException => StatusCodes.Status404NotFound,
            ArgumentException => StatusCodes.Status400BadRequest,
            InvalidOperationException => StatusCodes.Status409Conflict,
            _ => StatusCodes.Status500InternalServerError
        };

        await Results.Problem(
            statusCode: context.Response.StatusCode,
            title: "OrderFlow request failed",
            detail: feature?.Error.Message).ExecuteAsync(context);
    });
});

```

11.4 Logs úteis

- Inclua OrderId e EventId em logs de criação e processamento.
- Não registre senha, token ou connection string.
- Use níveis adequados: Information para fluxo normal, Warning para situação recuperável e Error para falha.
- No Docker e Kubernetes, escreva logs no console; a plataforma coleta stdout/stderr.

11.5 Definição de pronto

- ☐ dotnet build sem warnings importantes.
- ☐ dotnet test com todos os testes passando.
- ☐ A API retorna Problem Details em erros conhecidos.
- ☐ Health check responde 200.
- ☐ Logs mostram OrderId e EventId.
- ☐ Nenhuma senha real foi commitada.

12. Kubernetes local

Deployments, Services, ConfigMap, Secret e persistência

12.1 Preparar imagens no GitHub Container Registry

A forma mais previsível de o cluster local obter as imagens é publicá-las no GitHub Container Registry (GHCR). Para um repositório público, as imagens podem ser públicas. Substitua SEU_USUARIO pelo usuário do GitHub em todos os manifests.

Build e push manual [PowerShell]

```
docker build -f src/OrderFlow.Api/Dockerfile \
  -t ghcr.io/SEU_USUARIO/orderflow-api:1.0.0 .

docker build -f src/OrderFlow.Worker/Dockerfile \
  -t ghcr.io/SEU_USUARIO/orderflow-worker:1.0.0 .

docker login ghcr.io
docker push ghcr.io/SEU_USUARIO/orderflow-api:1.0.0
docker push ghcr.io/SEU_USUARIO/orderflow-worker:1.0.0
```

12.2 Namespace, ConfigMap e Secret

k8s/00-base.yaml [YAML]

```
apiVersion: v1
kind: Namespace
metadata:
  name: orderflow
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: orderflow-config
  namespace: orderflow
data:
  POSTGRES_DB: orderflow
  POSTGRES_USER: orderflow
  RABBITMQ_HOST: rabbitmq
  RABBITMQ_EXCHANGE: orderflow.events
  RABBITMQ_QUEUE: orderflow.order-created
---
apiVersion: v1
kind: Secret
metadata:
  name: orderflow-secrets
  namespace: orderflow
type: Opaque
stringData:
  POSTGRES_PASSWORD: orderflow123
  RABBITMQ_USER: orderflow
  RABBITMQ_PASSWORD: orderflow123
```

Atenção

stringData facilita o laboratório, mas o Secret não é criptografia por si só. Em produção, restrinja acesso ao cluster e use integração com um gerenciador de segredos.

12.3 PostgreSQL com PVC

k8s/10-postgres.yaml [YAML]

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: orderflow
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 2Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  namespace: orderflow
spec:
  replicas: 1
  selector:
    matchLabels: { app: postgres }
  template:
    metadata:
      labels: { app: postgres }
    spec:
      containers:
        - name: postgres
          image: postgres:18-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              valueFrom:
                configMapKeyRef: { name: orderflow-config, key: POSTGRES_DB }
            - name: POSTGRES_USER
              valueFrom:
                configMapKeyRef: { name: orderflow-config, key: POSTGRES_USER }
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef: { name: orderflow-secrets, key: POSTGRES_PASSWORD }
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "orderflow", "-d", "orderflow"]
            initialDelaySeconds: 5
            periodSeconds: 5
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "512Mi" }
      volumes:
        - name: data
          persistentVolumeClaim: { claimName: postgres-pvc }
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: orderflow
spec:
  selector: { app: postgres }
  ports:
    - port: 5432
      targetPort: 5432
```

Para aprendizado, um Deployment com PVC é suficiente. Em produção, prefira PostgreSQL gerenciado ou avalie StatefulSet e estratégia real de backup.

12.4 RabbitMQ

k8s/20-rabbitmq.yaml [YAML]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq
  namespace: orderflow
spec:
```

```

replicas: 1
selector:
  matchLabels: { app: rabbitmq }
template:
  metadata:
    labels: { app: rabbitmq }
  spec:
    containers:
      - name: rabbitmq
        image: rabbitmq:4-management-alpine
        ports:
          - { name: amqp, containerPort: 5672 }
          - { name: management, containerPort: 15672 }
        env:
          - name: RABBITMQ_DEFAULT_USER
            valueFrom:
              secretKeyRef: { name: orderflow-secrets, key: RABBITMQ_USER }
          - name: RABBITMQ_DEFAULT_PASS
            valueFrom:
              secretKeyRef: { name: orderflow-secrets, key: RABBITMQ_PASSWORD }
        readinessProbe:
          exec:
            command: ["rabbitmq-diagnostics", "-q", "ping"]
          initialDelaySeconds: 15
          periodSeconds: 10
    ---
apiVersion: v1
kind: Service
metadata:
  name: rabbitmq
  namespace: orderflow
spec:
  selector: { app: rabbitmq }
  ports:
    - { name: amqp, port: 5672, targetPort: 5672 }
    - { name: management, port: 15672, targetPort: 15672 }

```

12.5 API

k8s/30-api.yaml [YAML]

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: orderflow-api
  namespace: orderflow
spec:
  replicas: 1
  selector:
    matchLabels: { app: orderflow-api }
  template:
    metadata:
      labels: { app: orderflow-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/SEU_USUARIO/orderflow-api:1.0.0
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
          env:
            - name: ASPNETCORE_URLS
              value: http://+:8080
            - name: ConnectionStrings__Postgres
              value: Host=postgres;Port=5432;Database=orderflow;Username=orderflow;Password=orderflow123
            - name: RabbitMq__Host
              valueFrom:
                configMapKeyRef: { name: orderflow-config, key: RABBITMQ_HOST }
            - name: RabbitMq__User
              valueFrom:
                secretKeyRef: { name: orderflow-secrets, key: RABBITMQ_USER }
            - name: RabbitMq__Password
              valueFrom:
                secretKeyRef: { name: orderflow-secrets, key: RABBITMQ_PASSWORD }
            - name: RabbitMq__Exchange
              valueFrom:
                configMapKeyRef: { name: orderflow-config, key: RABBITMQ_EXCHANGE }
            - name: RabbitMq__Queue
              valueFrom:
                configMapKeyRef: { name: orderflow-config, key: RABBITMQ_QUEUE }
          readinessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 10

```

```

    periodSeconds: 5
  livenessProbe:
    httpGet: { path: /health, port: 8080 }
    initialDelaySeconds: 20
    periodSeconds: 10
  resources:
    requests: { cpu: "100m", memory: "128Mi" }
    limits: { cpu: "500m", memory: "512Mi" }
---
apiVersion: v1
kind: Service
metadata:
  name: orderflow-api
  namespace: orderflow
spec:
  selector: { app: orderflow-api }
  ports:
    - port: 8080
      targetPort: 8080

```

Melhoria necessária

No exemplo acima, a connection string contém a senha por simplicidade visual. No seu repositório, monte a connection string a partir de variáveis ou use um Secret com a string completa.

12.6 Worker

k8s/40-worker.yaml [YAML]

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: orderflow-worker
  namespace: orderflow
spec:
  replicas: 1
  selector:
    matchLabels: { app: orderflow-worker }
  template:
    metadata:
      labels: { app: orderflow-worker }
    spec:
      containers:
        - name: worker
          image: ghcr.io/SEU_USUARIO/orderflow-worker:1.0.0
          imagePullPolicy: Always
          env:
            - name: ConnectionStrings_Postgres
              value: Host=postgres;Port=5432;Database=orderflow;Username=orderflow;Password=orderflow123
            - name: RabbitMq_Host
              value: rabbitmq
            - name: RabbitMq_User
              valueFrom:
                secretKeyRef: { name: orderflow-secrets, key: RABBITMQ_USER }
            - name: RabbitMq_Password
              valueFrom:
                secretKeyRef: { name: orderflow-secrets, key: RABBITMQ_PASSWORD }
            - name: RabbitMq_Exchange
              value: orderflow.events
            - name: RabbitMq_Queue
              value: orderflow.order-created
          resources:
            requests: { cpu: "50m", memory: "96Mi" }
            limits: { cpu: "300m", memory: "256Mi" }

```

12.7 Aplicar e acompanhar

Deploy [PowerShell]

```

kubectl config use-context docker-desktop
kubectl apply -f k8s/00-base.yaml
kubectl apply -f k8s/10-postgres.yaml
kubectl apply -f k8s/20-rabbitmq.yaml
kubectl apply -f k8s/30-api.yaml
kubectl apply -f k8s/40-worker.yaml

kubectl get all -n orderflow
kubectl get pvc -n orderflow
kubectl rollout status deployment/orderflow-api -n orderflow

```

```
kubectl logs -f deployment/orderflow-api -n orderflow
kubectl logs -f deployment/orderflow-worker -n orderflow
```

12.8 Acessar com port-forward

Acesso local [PowerShell]

```
kubectl port-forward service/orderflow-api 8080:8080 -n orderflow
# em outro terminal:
kubectl port-forward service/rabbitmq 15672:15672 -n orderflow
```

12.9 Escalabilidade e SignalR

Você pode escalar o Worker porque as mensagens são divididas entre consumidores. Já a API com SignalR exige cuidado: uma conexão fica presa a um Pod. Para o MVP, mantenha uma réplica da API. Como evolução, use sticky sessions e um backplane Redis ou um serviço gerenciado de SignalR.

Escalar o Worker [PowerShell]

```
kubectl scale deployment/orderflow-worker --replicas=3 -n orderflow
kubectl get pods -n orderflow -w
```

- ☐ Todos os Pods ficam Running e Ready.
- ☐ O PVC do PostgreSQL está Bound.
- ☐ O port-forward abre a aplicação.
- ☐ Criar pedido gera log no Worker.
- ☐ Excluir um Pod da API faz o Deployment criar outro.
- ☐ Escalar o Worker cria múltiplos consumidores.

13. CI com GitHub Actions

Build, testes e publicação de imagens no GHCR

13.1 Workflow

.github/workflows/ci.yml [YAML]

```
name: ci

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

permissions:
  contents: read
  packages: write

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'
      - run: dotnet restore
      - run: dotnet build --configuration Release --no-restore
      - run: dotnet test --configuration Release --no-build

  docker:
    if: github.event_name == 'push'
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
```

```

with:
  registry: ghcr.io
  username: ${github.actor}
  password: ${secrets.GITHUB_TOKEN}
- uses: docker/build-push-action@v6
  with:
    context: .
    file: src/OrderFlow.Api/Dockerfile
    push: true
    tags: ghcr.io/${github.repository_owner}/orderflow-api:latest
- uses: docker/build-push-action@v6
  with:
    context: .
    file: src/OrderFlow.Worker/Dockerfile
    push: true
    tags: ghcr.io/${github.repository_owner}/orderflow-worker:latest

```

13.2 O que o pipeline prova

- O código compila em uma máquina limpa.
- Os testes são executados em cada Pull Request.
- As imagens Docker são reproduzíveis.
- O Kubernetes pode usar imagens publicadas automaticamente.
- O repositório apresenta práticas de integração contínua.

13.3 Estratégia simples de commits

Prefixo	Exemplo
chore	chore: create solution and tooling
feat	feat: add product CRUD
fix	fix: prevent negative stock
test	test: cover order total calculation
docs	docs: add architecture diagram
ci	ci: publish images to GHCR

14. Transformando em projeto de portfólio

README, demonstração e publicação no LinkedIn

14.1 Estrutura do README

Modelo de README.md [Markdown]

```

# OrderFlow
Plataforma de pedidos com chat em tempo real e processamento assíncrono.

## Demonstração
GIF ou vídeo de 60 a 90 segundos.

## Arquitetura
Diagrama e explicação dos três fluxos: HTTP, SignalR e RabbitMQ.

## Tecnologias
.NET 10, ASP.NET Core, EF Core, PostgreSQL, SignalR,
RabbitMQ, Docker Compose, Kubernetes e GitHub Actions.

## Funcionalidades
Lista do MVP com screenshots.

## Como executar com Docker Compose
Comandos e URLs.

## Como executar no Kubernetes
Pré-requisitos e kubectl apply.

## Decisões técnicas
Por que SignalR, RabbitMQ, Worker, idempotência e uma réplica da API.

```


Próximos passos
JWT, Outbox, DLQ, OpenTelemetry, Redis backplane e Helm.

14.2 Screenshots que valem a pena

- Dashboard com lista de produtos e pedidos.
- Duas janelas trocando mensagens na mesma sala.
- Mudança de status aparecendo em tempo real.
- Swagger com endpoints organizados.
- RabbitMQ Management mostrando exchange, fila e consumidor.
- Docker Desktop com os containers em execução.
- Kubernetes mostrando Pods, Services e Deployments.
- GitHub Actions com pipeline verde.

14.3 Roteiro para vídeo de 75 segundos

Tempo	Demonstração
0-10s	Apresente problema, stack e arquitetura.
10-25s	Cadastre produto e cliente; crie pedido.
25-40s	Abra duas janelas e envie mensagens.
40-50s	Mude o status e mostre atualização instantânea.
50-60s	Mostre RabbitMQ e log do Worker.
60-70s	Mostre Docker/Kubernetes e pipeline.
70-75s	Exiba o link do GitHub e próximos passos.

14.4 Texto-base para LinkedIn

Sugestão de publicação

Concluí o OrderFlow, um projeto de estudo em C#/.NET voltado a arquitetura de backend. A aplicação possui CRUD de produtos e clientes, criação e acompanhamento de pedidos, chat por pedido com SignalR e processamento assíncrono com RabbitMQ e Worker Service. O ambiente roda com PostgreSQL, Docker Compose e Kubernetes local, com pipeline de build, testes e publicação de imagens no GitHub Actions. O principal aprendizado foi separar corretamente os fluxos síncrono, em tempo real e assíncrono, além de lidar com persistência, idempotência, health checks e configuração por ambiente. Repositório e demonstração: [adicione o link].

14.5 Palavras-chave

#dotnet #csharp #aspnetcore #docker #kubernetes #postgresql #rabbitmq #signalr #backend #softwarearchitecture #devops #opensource

Evite um post genérico

Mostre uma decisão concreta, por exemplo: “Usei grupos do SignalR para isolar cada pedido e índice único por EventId para garantir idempotência do Worker.” Isso demonstra entendimento.

15. Evoluções avançadas

Como transformar o laboratório em uma arquitetura mais próxima de produção

Evolução	O que você aprende
JWT e autorização	Proteger endpoints e limitar acesso ao pedido.
Outbox Pattern	Publicar eventos com consistência entre banco e broker.
Dead Letter Queue	Isolar mensagens que falham repetidamente.
Retry com backoff	Tratar falhas temporárias sem sobrecarregar serviços.
Redis backplane	Escalar SignalR para várias instâncias.
OpenTelemetry	Rastrear uma operação entre API, RabbitMQ e Worker.
Testcontainers	Criar testes de integração reproduzíveis.
Helm	Parametrizar e empacotar manifests Kubernetes.
Ingress + TLS	Expor a aplicação por domínio e HTTPS.
PostgreSQL gerenciado	Separar banco stateful do cluster de aplicações.

15.1 Ordem recomendada

1. Primeiro termine o MVP funcionando em Docker Compose.
2. Depois adicione testes e tratamento de erros.
3. Em seguida execute no Kubernetes com uma réplica da API.
4. Implemente Outbox e DLQ antes de adicionar mais microsserviços.
5. Adicione autenticação antes de publicar uma demonstração acessível pela internet.
6. Use observabilidade para provar o fluxo completo.

15.2 Desafios práticos

- ☐ Criar busca e paginação de produtos.
- ☐ Adicionar cupom de desconto com regras de validade.
- ☐ Criar evento order.status-changed e outro consumidor.
- ☐ Adicionar DLQ e uma tela para reprocessar mensagens.
- ☐ Implementar JWT e vincular usuário a cliente.
- ☐ Impedir que um cliente entre na sala de outro pedido.
- ☐ Adicionar upload de comprovante em armazenamento de objetos.
- ☐ Adicionar métricas de pedidos por status.
- ☐ Criar Helm Chart para todo o ambiente.
- ☐ Executar teste de carga no endpoint de criação de pedidos.

16. Solução de problemas

Erros comuns no Windows, Docker, RabbitMQ, SignalR e Kubernetes

Sintoma	Verificação e correção
docker não conecta	Abra Docker Desktop, confirme WSL 2 e execute wsl --update.
Porta 5432 ocupada	Pare PostgreSQL local ou altere a porta externa para 5433:5432.
API não encontra postgres	Dentro do Compose use Host=postgres, não localhost.
RabbitMQ connection refused	Aguarde o health check e confirme usuário/senha.
guest não conecta entre containers	Crie usuário próprio; guest normalmente é limitado ao localhost.
Migration falha	Confira startup-project, project e connection string.

Sintoma	Verificação e correção
SignalR desconecta	Confira URL do Hub, logs do navegador e proxy/HTTPS.
Mensagem aparece em sala errada	Revise o nome do group e sempre inclua OrderId.
Worker repete efeito	Adicione idempotência por EventId e índice único.
ImagePullBackOff	Confira nome/tag da imagem e visibilidade no GHCR.
CrashLoopBackOff	Use kubectl logs e kubectl describe pod.
Pod não fica Ready	Revise readiness probe e dependências.
Dados somem no Compose	Confirme volume e não execute down -v.
Dados somem no Kubernetes	Confirme PVC Bound e volumeMount correto.

16.1 Comandos de emergência

Diagnóstico [PowerShell]

```
# Docker
docker compose ps
docker compose logs -f --tail 200
docker stats
docker system df

# Kubernetes
kubectl get pods -n orderflow -o wide
kubectl describe pod NOME_DO_POD -n orderflow
kubectl logs NOME_DO_POD -n orderflow --previous
kubectl get events -n orderflow --sort-by=.metadata.creationTimestamp
kubectl rollout restart deployment/orderflow-api -n orderflow
kubectl delete pod NOME_DO_POD -n orderflow

# Limpar o laboratório Kubernetes
kubectl delete namespace orderflow
```

Método de diagnóstico

Leia o erro de baixo para cima: aplicação, configuração, rede, dependência e infraestrutura. Evite apagar tudo antes de observar logs e eventos, porque você perde a evidência da causa.

17. Checklist final e plano de estudo

Uma sequência para concluir e publicar

17.1 Checklist técnico

- ☐ Solução .NET separada em Domain, Application, Infrastructure, API e Worker.
- ☐ PostgreSQL com migrations e índices.
- ☐ CRUD de produtos e clientes.
- ☐ Criação de pedido com itens, total e estoque.
- ☐ Evento order.created publicado no RabbitMQ.
- ☐ Worker com ack manual, logs e idempotência.
- ☐ Chat SignalR por grupo de pedido.
- ☐ Histórico do chat persistido.
- ☐ Mudança de status enviada em tempo real.
- ☐ Dockerfiles multi-stage.
- ☐ docker compose up -d inicia tudo.
- ☐ Testes unitários passando.
- ☐ Health check configurado.
- ☐ Manifests Kubernetes organizados.

- ☐ Pods, Services, ConfigMap, Secret e PVC funcionando.
- ☐ GitHub Actions verde e imagens no GHCR.

17.2 Checklist de portfólio

- ☐ README com proposta, arquitetura e instruções.
- ☐ Diagrama legível no início do README.
- ☐ Screenshots sem dados pessoais.
- ☐ Vídeo curto mostrando fluxo completo.
- ☐ Issues ou Project Board com as etapas.
- ☐ Commits pequenos e descritivos.
- ☐ Licença MIT ou outra licença escolhida.
- ☐ Release v1.0.0 criada no GitHub.
- ☐ Texto do LinkedIn explica aprendizados, não apenas tecnologias.

17.3 Plano de quatro semanas

Semana	Foco	Entrega
1	.NET, domínio, EF Core e PostgreSQL	CRUD + migrations
2	Pedidos, RabbitMQ e Worker	Evento processado com idempotência
3	SignalR, interface e Docker Compose	Demonstração local completa
4	Testes, Kubernetes, CI e documentação	Release pública e post

17.4 Definição de sucesso

Você terá concluído o estudo quando conseguir explicar, sem ler o código, por que cada componente existe, o que acontece quando ele falha e como observar o comportamento do sistema. A capacidade de explicar as decisões é tão importante quanto a aplicação funcionando.

A. Comandos rápidos

Folha de consulta para o dia a dia

Cheat sheet [PowerShell]

```
# .NET
dotnet restore
dotnet build
dotnet test
dotnet run --project src/OrderFlow.Api
dotnet run --project src/OrderFlow.Worker

# EF Core
dotnet ef migrations add NomeDaMigration \
  --project src/OrderFlow.Infrastructure \
  --startup-project src/OrderFlow.Api
dotnet ef database update \
  --project src/OrderFlow.Infrastructure \
  --startup-project src/OrderFlow.Api

# Docker Compose
docker compose up -d --build
docker compose ps
docker compose logs -f api worker
docker compose down

# Kubernetes
kubectl get all -n orderflow
kubectl apply -f k8s/
kubectl logs -f deployment/orderflow-worker -n orderflow
kubectl port-forward service/orderflow-api 8080:8080 -n orderflow
kubectl delete namespace orderflow
```

B. Referências oficiais

Documentação usada como base e caminho para aprofundamento

1. Microsoft - .NET support policy

<https://dotnet.microsoft.com/platform/support/policy>

2. Microsoft Learn - ASP.NET Core Web API

<https://learn.microsoft.com/aspnet/core/web-api/>

3. Microsoft Learn - ASP.NET Core SignalR overview

<https://learn.microsoft.com/aspnet/core/signalr/introduction>

4. Microsoft Learn - SignalR hubs

<https://learn.microsoft.com/aspnet/core/signalr/hubs>

5. Microsoft Learn - Entity Framework Core

<https://learn.microsoft.com/ef/core/>

6. Docker Docs - Install Docker Desktop on Windows

<https://docs.docker.com/desktop/setup/install/windows-install/>

7. Docker Docs - WSL 2 backend

<https://docs.docker.com/desktop/features/wsl/>

8. Docker Docs - Kubernetes in Docker Desktop

<https://docs.docker.com/desktop/use-desktop/kubernetes/>

9. Kubernetes - Concepts

<https://kubernetes.io/docs/concepts/>

10. Kubernetes - Deployments

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

11. Kubernetes - Services

<https://kubernetes.io/docs/concepts/services-networking/service/>

12. Kubernetes - ConfigMaps

<https://kubernetes.io/docs/concepts/configuration/configmap/>

13. Kubernetes - Secrets

<https://kubernetes.io/docs/concepts/configuration/secret/>

14. RabbitMQ - .NET client API guide

<https://www.rabbitmq.com/client-libraries/dotnet-api-guide>

15. RabbitMQ - .NET tutorials

<https://www.rabbitmq.com/tutorials>

16. PostgreSQL - Current documentation

<https://www.postgresql.org/docs/current/>

Observação de versão

Tecnologias mudam. Use a documentação oficial para ajustar versões de imagens, pacotes e comandos. O projeto foi planejado para .NET 10 LTS e ferramentas disponíveis em julho de 2026.

FIM - agora transforme cada capítulo em um commit.